

0.87. Agrandir une lettre : texteGrand

`texteGrand(x, y, "A", 2)` : le A trois fois plus grand, sans rien dessiner. Chaque pixel devient un carré de 3×3 pixels ; les proportions sont gardées.

1. Ce qu'il faut comprendre

Une lettre plus grande, sans la dessiner. La Game Boy n'a pas de zoom, mais la console peut **calculer** une lettre agrandie à partir de la police : chaque pixel devient un carré.

Ce qui est nouveau ici : `texteGrand(x, y, "TEXTE", taille)`. La **taille** va de **0 à 10**. **0, c'est la lettre normale** ; chaque cran de plus l'agrandit **une fois de plus** : $1 \rightarrow 2$ fois, $2 \rightarrow 3$ fois... $10 \rightarrow 11$ fois. Avec `2`, chaque pixel de la lettre devient un carré de **3×3 pixels**. La lettre garde **exactement ses proportions**.

La place qu'elle prend : une lettre normale (taille 0) tient dans **1 case** (8×8 pixels). À la taille 2, elle est 3 fois plus grande et prend **$3 \times 3 = 9$ cases** (24×24 pixels). À la taille $*n*$, une lettre prend $*n + 1*$ cases de large et $*n + 1*$ de haut.

Le texte et la taille s'écrivent en clair (`"A"`, `2`), comme pour `ms()` : le compilateur fabrique les tuiles agrandies **avant le jeu**, et seulement celles dont le programme a besoin.

Ce qu'elle coûte : quelques tuiles de plus dans la cartouche. Deux tuiles identiques (souvent toutes pleines aux grandes tailles) ne sont fabriquées qu'une fois.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1170 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          55 o ← la console, à l'allumage — la mise en route de la console
texte_grand_0()    146 o ← main() — ta fonction
main()             18 o ← démarrage — ta fonction
VBlank             8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
AdresseCase        10 o ← poser() — trouve l'adresse d'une case de l'écran
CopierTuiles       18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage — vide les deux cartes de l'écran
Tuiles            848 o ← CopierTuiles — la police entière, et 9 tuiles dessinées
GRAND_TABLE_0_0_0   3 o ← poser() — une table gravée
GRAND_TABLE_0_1_1   3 o ← poser() — une table gravée
GRAND_TABLE_0_2_2   3 o ← poser() — une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond,
EffacerPanneau, LireManette, DonneesTransfert, Notes
```

Fonctions du langage appelées :

```
poser()          × 3    91 o sur place, dans texte_grand_0() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (3 o),  
GRAND_TABLE_0_1_1 (3 o), GRAND_TABLE_0_2_2 (3 o)  
image()          × 1    3 o sur place, dans main() + AttendreImage (28 o)  
Tes fonctions appelées :  
texte_grand_0() × 1    3 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

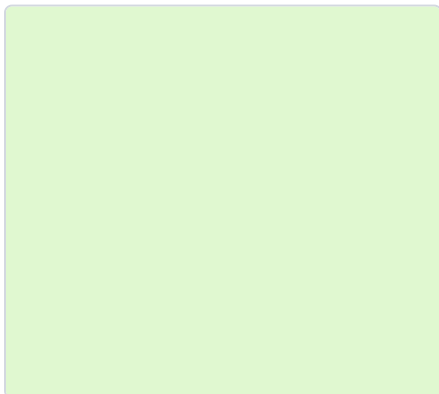
```
#include <texteGrand> // écrit un texte agrandi, taille de 0 à 10
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

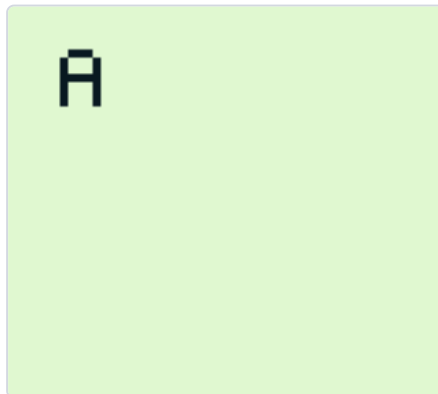
```
int main() {  
    // La ligne, morceau par morceau :  
    //  
    //  texteGrand(2, 2, "A", 2);  
    //          | | | |  
    //          | | | +-- la TAILLE (de 0 à 10) : 2 → 3 fois plus grand  
    //          | | +----- le texte, entre guillemets  
    //          +----- la case en haut à gauche du grand A : (2, 2)  
    //  
    // Chaque pixel du A devient un carré de 3 × 3 pixels :  
    //  
    //  le A normal (8 × 8)      le A × 3 (24 × 24 pixels = 3 × 3 cases)  
    //  ..■■■■..                .....■■■■■■■■■■.....  
    //  ■■■■■■                .....■■■■■■■■■■.....  
    //  ■■■■■■                .....■■■■■■■■■■.....  
    //  ■■■■■■                .....■■■■■■■■■■.....  
    //  ■■■■■■                .....■■■■■■■■■■.....  
    //  ...                    ...  
    texteGrand(2, 2, "A", 2);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un grand A, trois fois plus grand qu'une lettre normale, en haut à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le grand A occupe ses 3×3 cases

✓ rien à côté

Ensuite : **0.87.1. Agrandir une lettre : `texteGrand` — toutes les tailles** — chapitre 0 : Avant tout. · `gameboy3` — écrire une cartouche Game Boy en C++